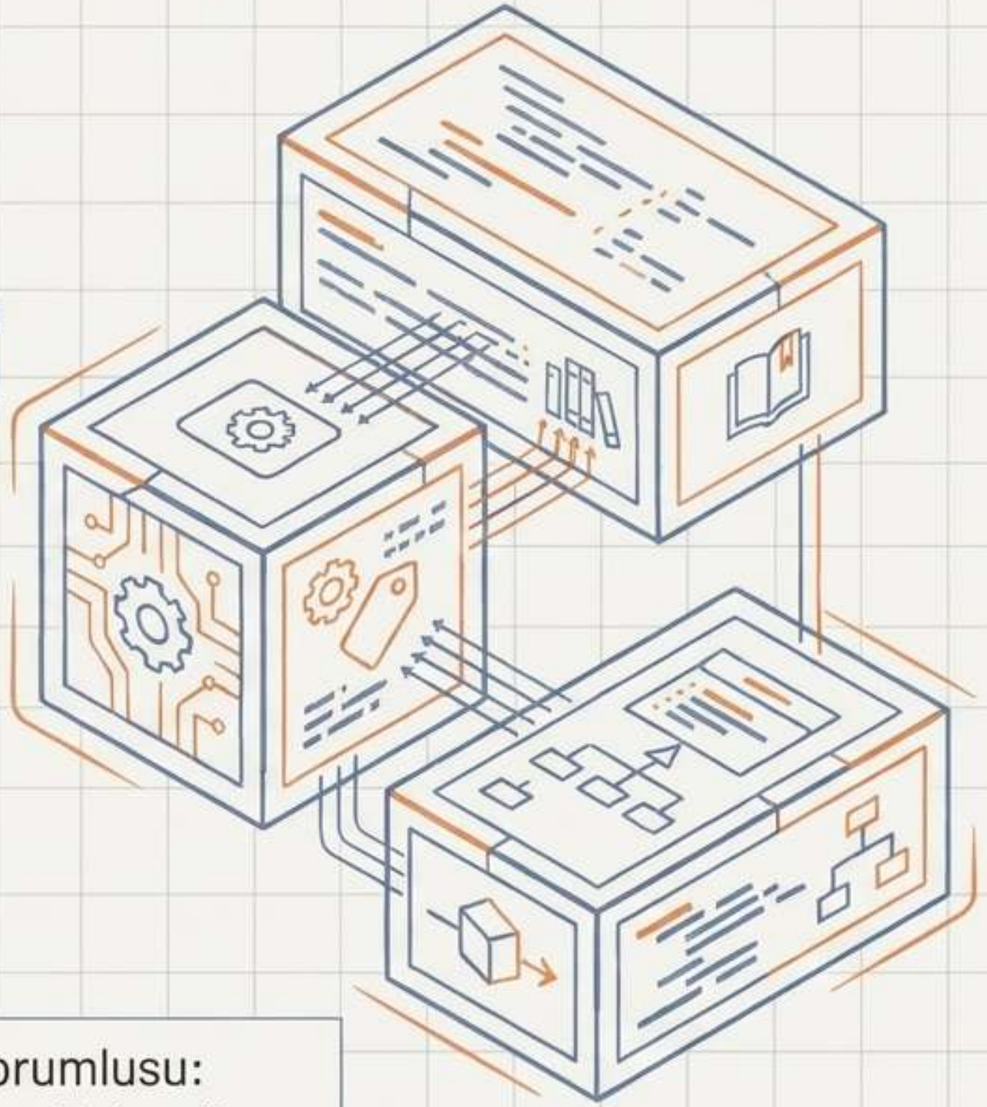
Temiz ve Hafif Kod: Dart 3.13 ile Modern Programlama Mimarisi

Bilişim Sistemleri 3. Sınıf İleri Programlama – Kütüphaneler, Metadata ve Sınıf Hiyerarşileri



2024481050	Eymen Zeycan İşleksoy
2024481004	Osman Ceran
2024481007	Emir Sevgili
2024481033	Murat İbrahim Polat

Ders Sorumlusu:
Dr. Mesut Polatgil

Gemini Notebook

Gündem: Mimarın Araç Kutusu



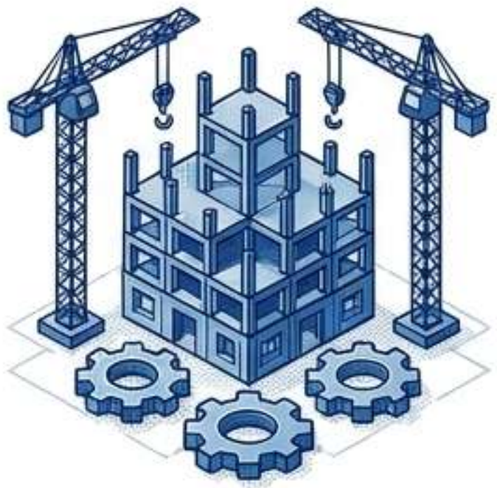
1. Organizasyon (Kütüphaneler)

- Modülerlik ve İçe Aktarmalar
- Gizlilik Sınırları (Privacy)
- Performans: Tembel Yükleme



2. İletişim (Metadatalar)

- Derleyici ile Konuşmak
- Kod Zekası ve Yönlendirme
- Özel Annotation'lar

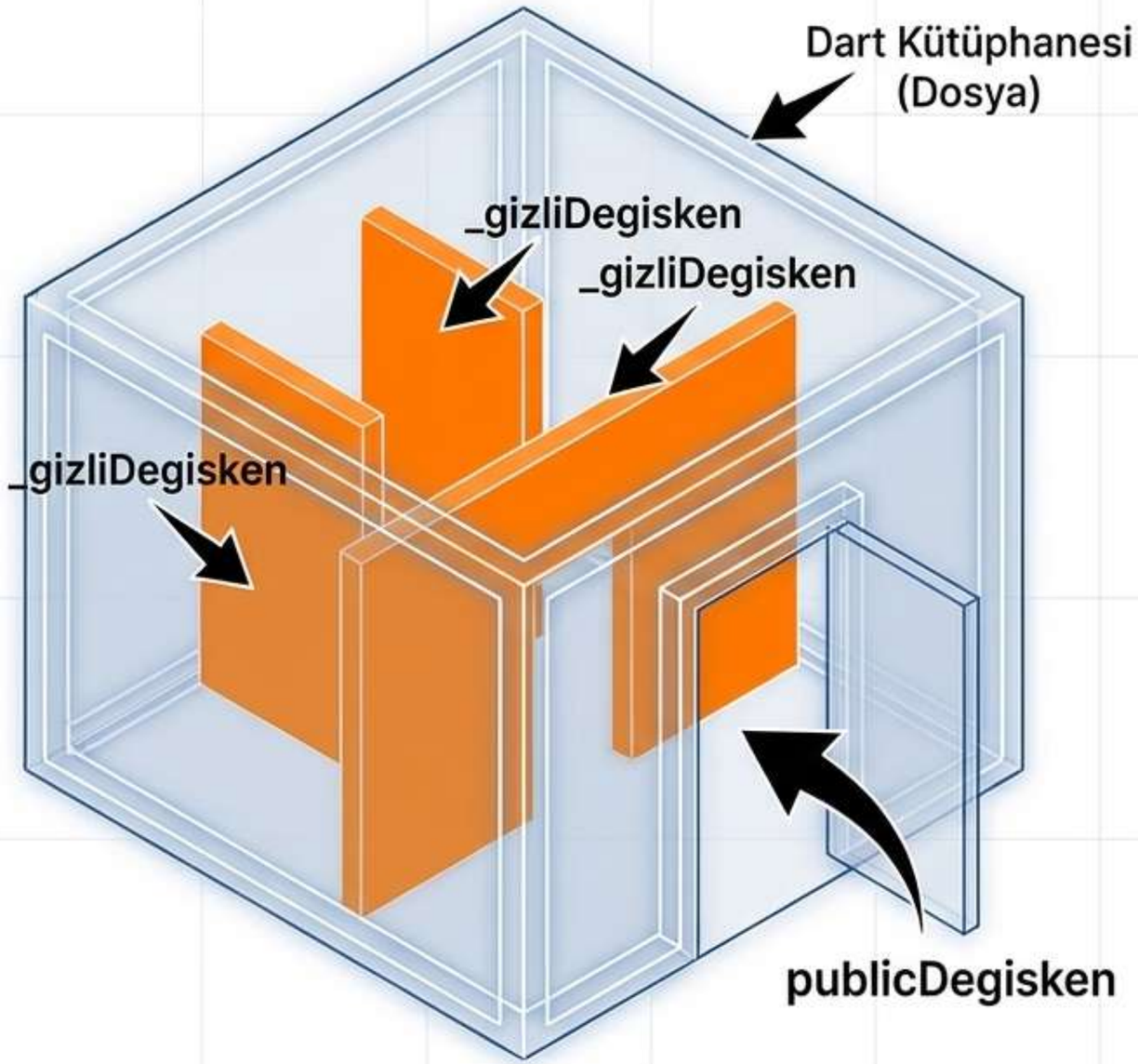


3. İnşa (Modern Sınıflar)

- Primary Constructors
- Mixin Evrimi
- Kapsamlı Sınıf Değiştiricileri

Gemini Notebook

Kütüphaneler ve Gizlilik (Privacy) Sınırları



- Dart dilinde public, private, protected anahtar kelimeleri yoktur.
- Her Dart dosyası bağımsız bir kütüphanedir.

Alt Çizgi Kuralı

Sadece _ ile başlayan tanımlayıcılar o kütüphanenin içinde yaşar ve dışarı sızamaz.

Mimari Avantaj

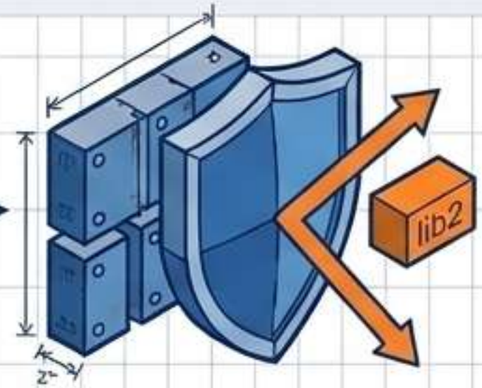
Bu basit yapı, Tree Shaking (ölü kod ayıklama) işlemini hızlandırır ve dinamik erişim yönetimini kolaylaştırır.

Gemini Notebook
Gemini Notebook

Akıllı İçe Aktarmalar: Kodu Filtrelemek

1. Çakışma Önleyici (Prefix)

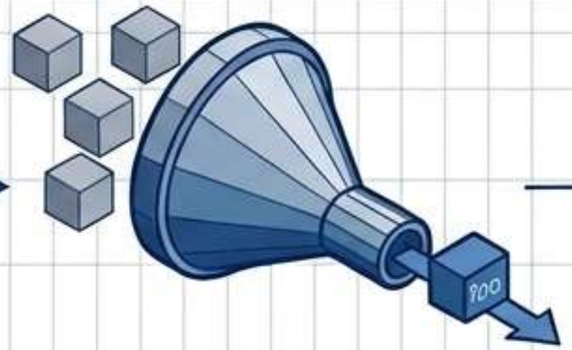
```
import 'package:lib2/lib2.dart'
as lib2;
```



Çakışma Önleyici (Prefix):
İsim çakışmalarını önlemek için kütüphaneye bir önek (namespace) atar.

2. Sadece İstenen (Allowlist)

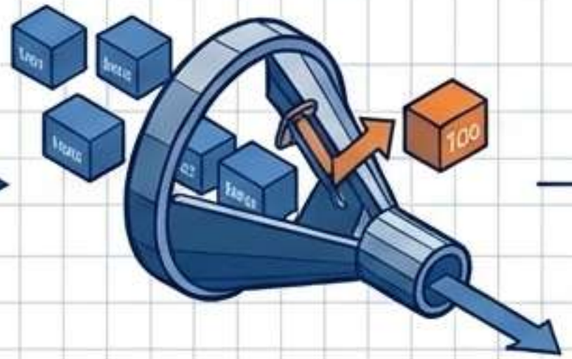
```
import 'package:lib1/lib1.dart'
show foo;
```



Sadece İstenen (Allowlist):
Sınırlı içe aktarma. Yalnızca belirtilen kısımları (foo) kod tabanına dahil eder.

3. Hariç Tutma (Blocklist)

```
import 'package:lib2/lib2.dart'
hide foo;
```



Hariç Tutma (Blocklist):
İstenmeyen kısımları kod tabanının dışında bırakır.

Gemini Notebook

Architectural Blueprint

Performans Odaklı Mimari: Tembel Yükleme (Deferred Loading)

```
import 'package:greetings/hello.dart' deferred as hello;
```



Başlangıç
Anında Yükleme

Kullanıcı Tetiklemesi
Etkileşim Bekleme

Ağdan Yükleme
Talep Üzerine İndirme

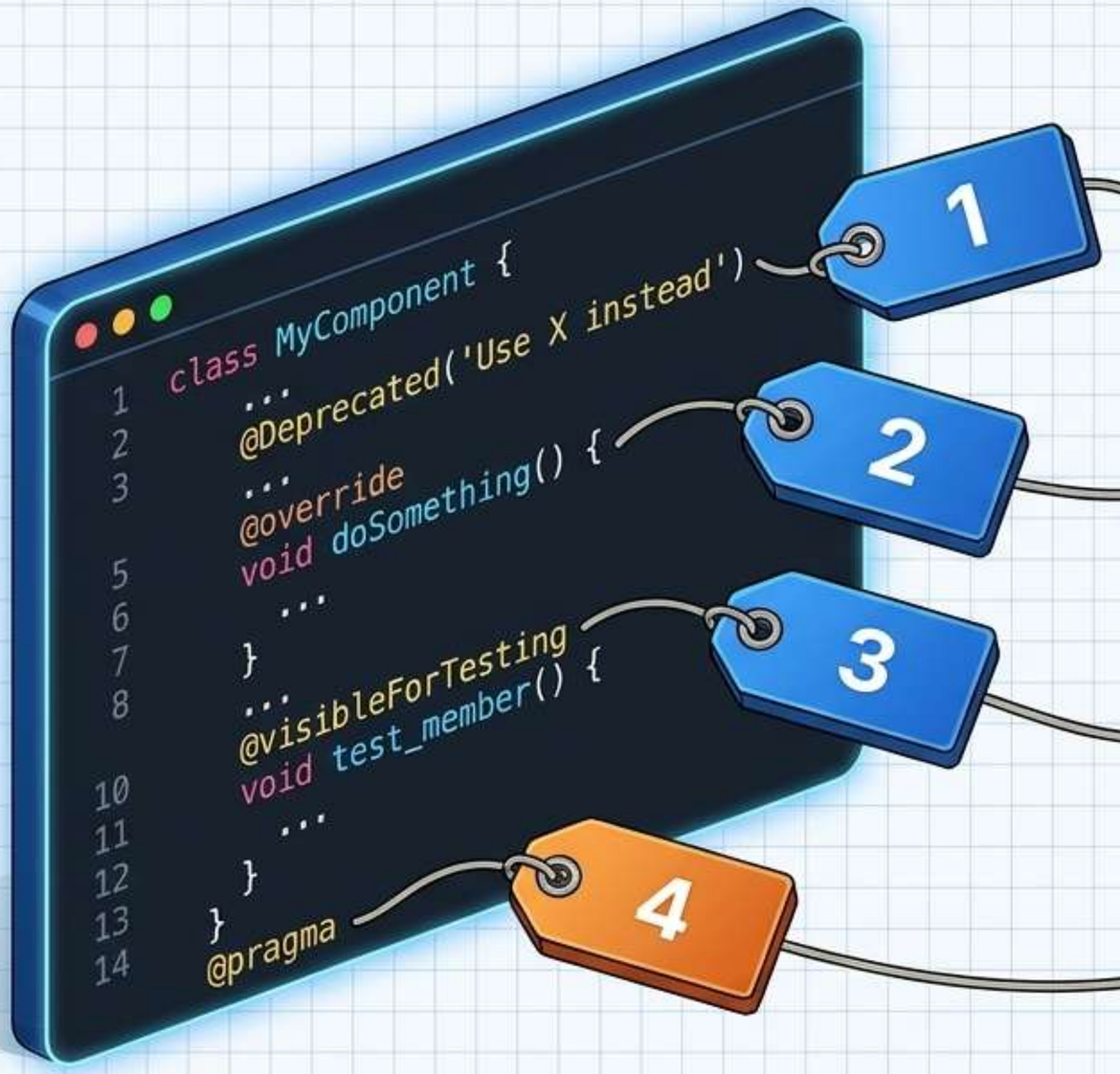
```
await hello.loadLibrary();
```

Mimari Karar (Ne zaman kullanılmalı?)
- Web uygulamasının ilk açılış süresini (TTI) kısaltmak.
- Nadir kullanılan ekranları ve diyalogları ayırmak.
- A/B testlerinde alternatif algoritmaları denemek.

Gemini Notebook

Metadata: Derleyici ile İletişim Kurmak

Metadata (@), çalışma zamanı kodunu değiştirmez; derleyiciye, analyzer'a ve diğer geliştiricilere statik bilgi enjekte eder.



`@Deprecated('Use X instead')`

Kodu tedavülden kaldırır, geliştiriciyi yeni metoda yönlendiren net bir mesaj ve uyarı verir.

`@override`

Üst sınıf metotlarının güvenle ezildiğini doğrular. Hata riskini sıfırlar.

`@visibleForTesting`

Üyeyi yalnızca test ortamında public yapar. Mimari sınırları katılaştırır.

`@pragma`

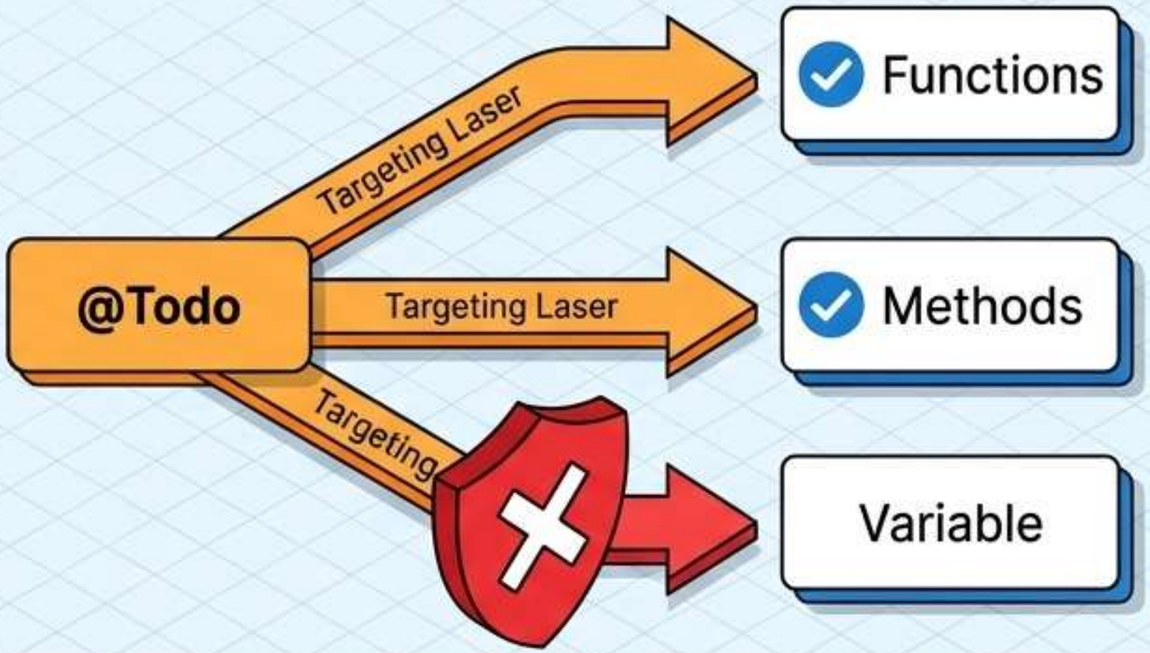
Derleyici ve analyzer gibi araçlara doğrudan, düşük seviyeli talimatlar gönderir.

Gemini
Gemini Notebook

Kendi Kuralını Yarat: Özel Annotation'lar

Dart'ta her const constuctor'a sahip sınıf, bir metadata etiketine dönüşebilir. Kendi kurallarınızı derleyiciye öğretebilirsiniz.

```dart
@Target({TargetKind.function, TargetKind.method})
class Todo {
  final String who;
  final String what;
  const Todo(this.who, this.what);
}
```



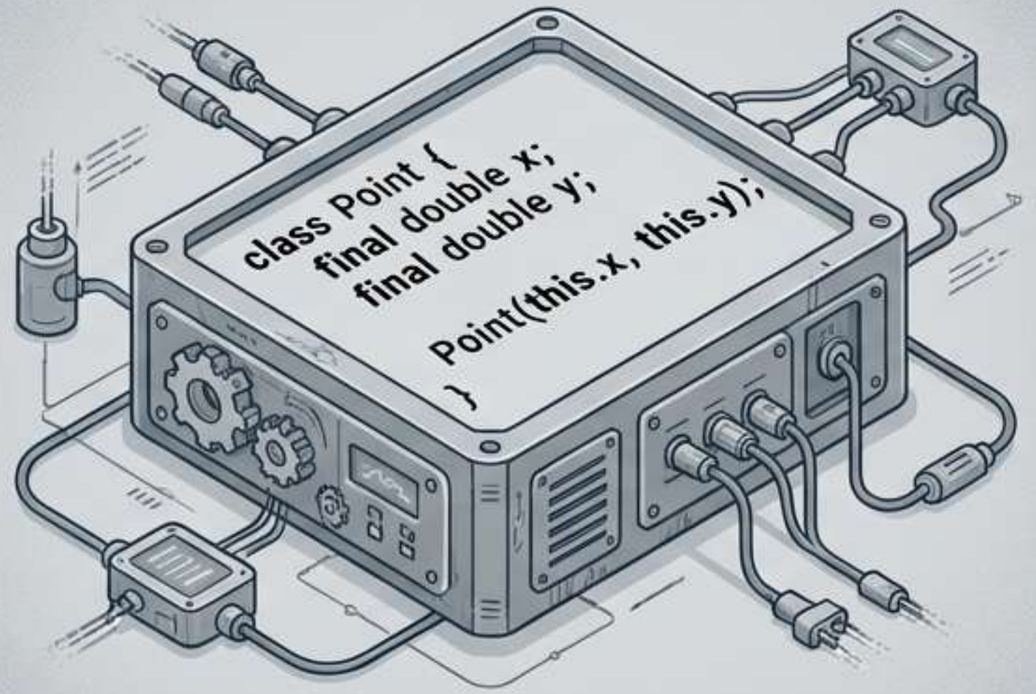
Aha Anı

Bu yapılandırma ile @Todo etiketi yanlış bir yere yazılırsa (örneğin bir sınıfın üstüne), analyzer anında mimari uyarı verir. Hatalar derleme öncesi yakalanır.

Gemini Notebook

Mimari Devrim: Primary Constructors

Geleneksel Sınıf Yapısı



Gereksiz tekrar (Boilerplate), uzun tanımlamalar.

Dart 3 - Primary Constructor



Sınıfı ve değişkenleri tek satırda tanımlama.

Mimari Etki: Çalışma zamanı (runtime) davranışını değiştirmez, ancak okunabilirliği mükemmelleştirir.

Gemini Notebook

Sabit (Const) Primary Constructor Kuralları

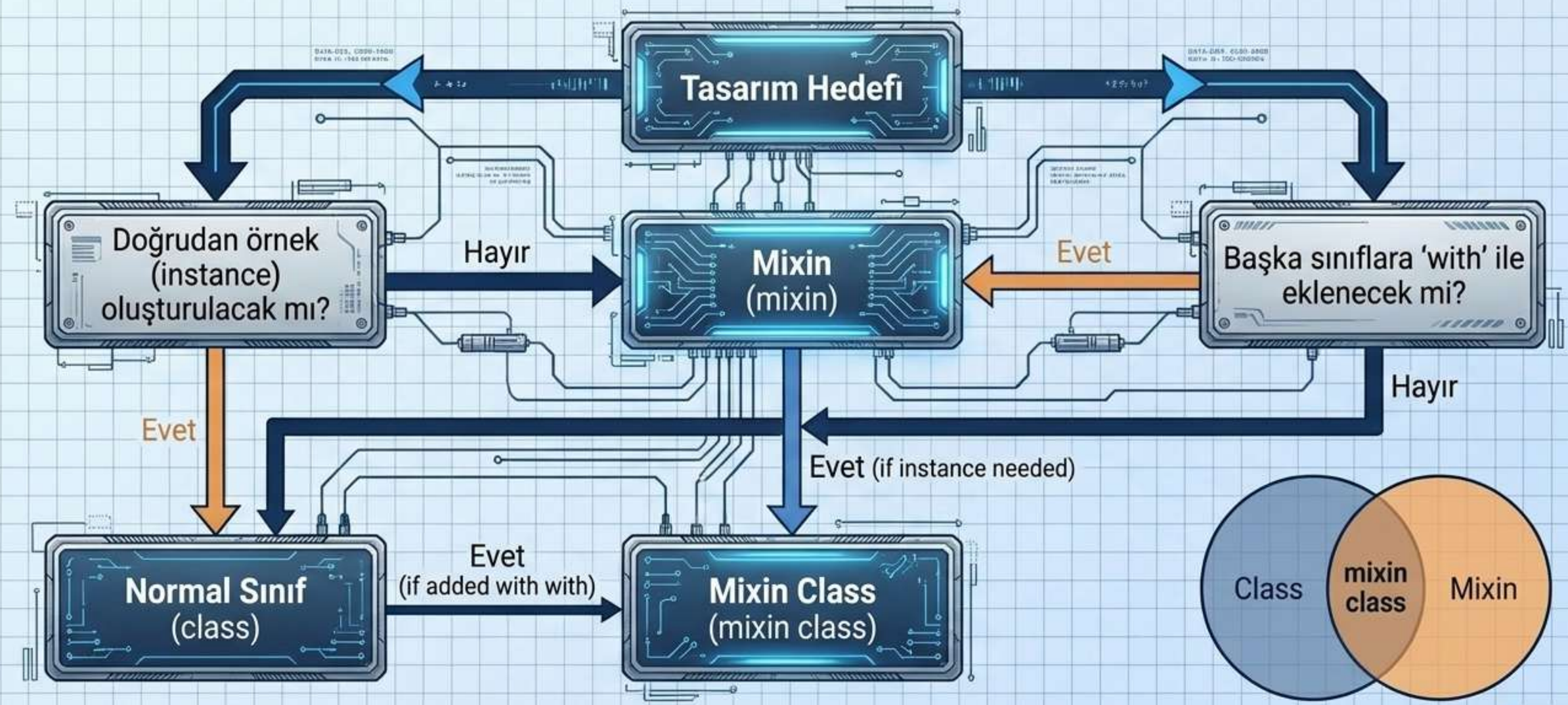
Sabit (const) sınıflar inşa ederken kurallar son derece katıdır:



Gemini Notebook

Mixin’lerin Evrimi ve Tasarım Kararları

Dart 3 ile birlikte mixin kuralları katılaştı. API tasarlarken kendinize iki soru sorun:



Gemini Notebook

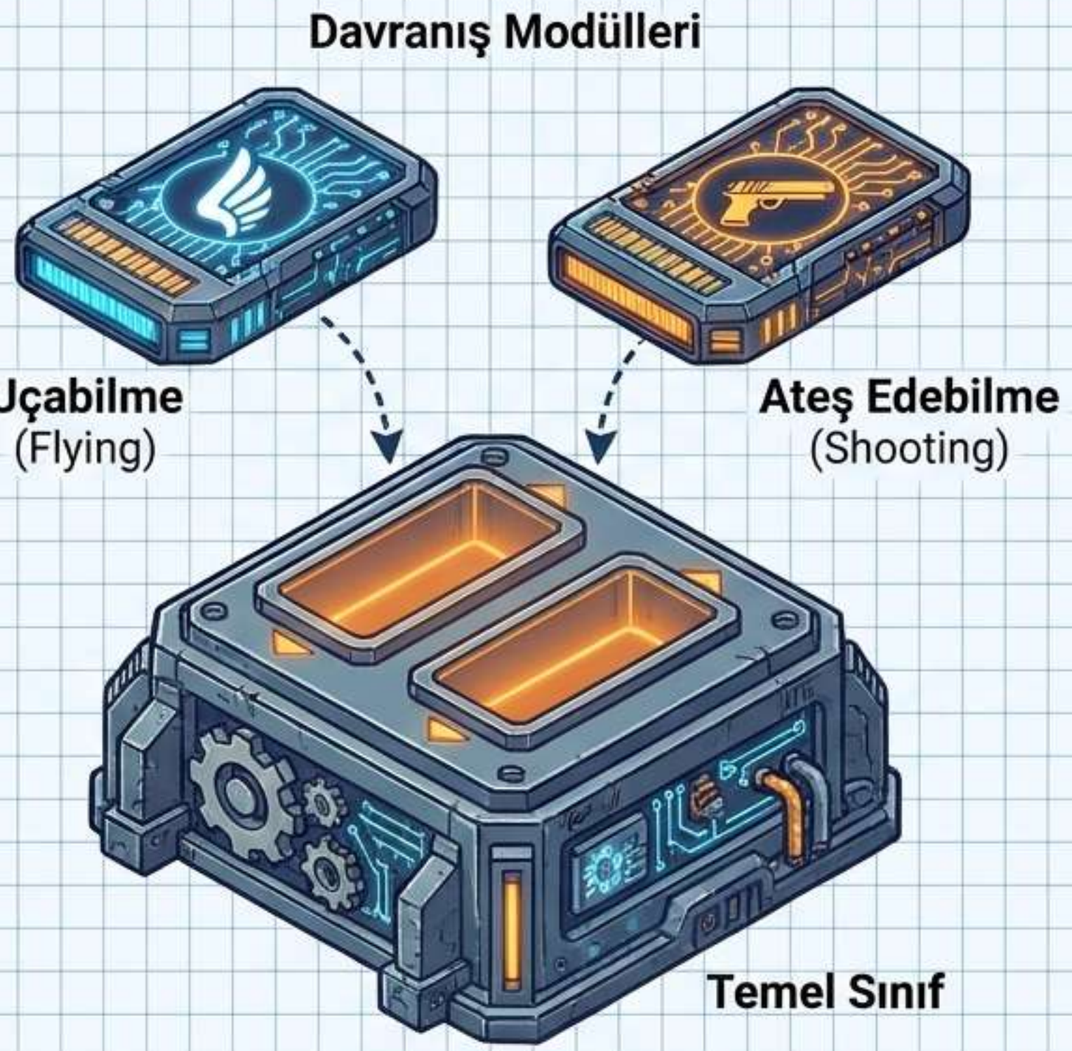
Mixin Kavramı: Modüler Davranış Paylaşımı

Problem

"Elmas Problemi (Diamond Problem)".

Geleneksel OOP dillerinde bir sınıfın birden fazla ebeveynden kalıtım (multiple inheritance) alması çatışmalara yol açar.

Hiyerarşi katı ve kırılgan hale gelir.



Çözüm

"Mixin Yaklaşımı"

Sınıf hiyerarşisini bozmadan, bağımsız yetenekleri (davranışları) istediğiniz sınıfa tak-çalıştır mantığıyla (composition) eklemenizi sağlar. Sınıflarınız yetenekleri miras almaz, yeteneklerle donatılır.

Gemini Notebook

Mixin Kod Anatomisi: 'with' ve 'on'

Deklarasyon:
Bağımsız ve yeniden kullanılabilir bir yetenek modülü tanımlar. Tek başına örneği (instance) oluşturulamaz.

Kısıtlama (Targeting):
Güvenlik duvarı. Bu mixin'in SADECE Performansci (veya alt sınıfları) tarafından kullanılabileceğini garanti eder.
mixin içindeki kod, Performansci sınıfının metotlarına güvenle erişebilir.

```dart
mixin Muzikal on Performansci {
  void sarkiSoyle() => print('Şarkı söyleniyor...');
}

class Dansci extends Performansci with Muzikal {
  // Dansci sınıfı artık sarkiSoyle() yeteneğine sahip!
}
```

Uygulama (Injection):
Yeteneği hedef sınıfa (Dansci) enjekte eder.
Sınıf hiyerarşisinde araya girerek, metotların doğru sırayla çağrılmasını (Linearization) sağlar.

Gemini Notebook

Gelişmiş Enum'lar (Enhanced Enums): Katı Durum Makineleri

Sabit Durumlar:

Önceden Tanımlı Kümeler: Sınıfın alabileceği tek ve kesin durumlar burada sabitlenir. Çalışma zamanında yeni instance oluşturulamaz.

Sınıf Yetenekleri:

Tıpkı Sınıflar Gibi: Dart 3 enum'ları kendi değişkenlerini, constructor'larını ve getter metotlarını barındırabilir.

```
enum AgDurumu {
  bagli(200, 'Bağlantı Başarılı'),
  kopuk(404, 'Sunucu Bulunamadı'),
  beklemede(102, 'İşlem Sürüyor');

  final int kod;
  final String mesaj;
  const AgDurumu(this.kod, this.mesaj);

  bool get isError => kod >= 400;
}
```

Eksiksizlik (Exhaustiveness):

Derleyici Garantisi: Switch yapılarında kullanıldığında tüm durumların (bağlı, kopuk, beklemede) işlendiğini derleme aşamasında (compile-time) garanti eder.

Gemini Notebook

Sentez: Dart 3 Sınıf Değiştiricileri (Modifiers) Matrisi

Not: Bu kurallar kütüphane dışındaki (farklı dosyalardaki) kullanımlar için geçerlidir.

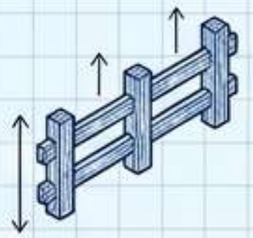
Hangi Değiştiriciyi Ne Zaman Kullanmalıyız?

Bildirim (Modifier)	Örnekle (Construct)?	Genişlet (Extend)?	Uygula (Implement)?	Mixin Ekle?
class	✓	✓	✗	✗
base class	✓	✓	✗	✗
interface class	✓	✗	✓	✗
final class	✓	✗	✗	✗
sealed class **Sadece aynı dosya içinde izin verilir	✗	✗	✗	✗
mixin	✗	✓	✓	✓

interface (Koruma)

Kullanıcıların arayüzü uygulamasına izin vermek ama sizin kodunuzun miras alınmasını engellemek için.

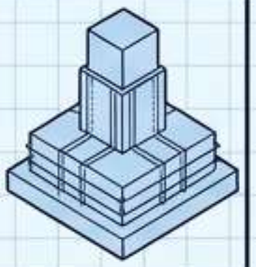
Sözleşmeyi dayatır.



base (Sağlamlaştırma)

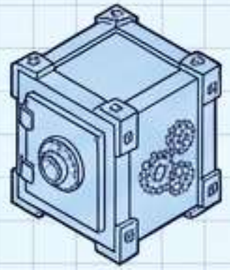
Kullanıcıların arayüzü yeniden yazmasını engeller.

Sınıf kodunuzun yeniden kullanımını (kalıtımı) zorunlu kılar.



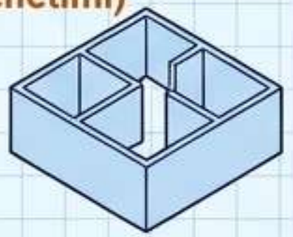
final (Tam Kilit)

Dışarıdan sınıfın genişletilmesini veya uygulanmasını tamamen engeller. Davranışın değiştirilemeyeceğinin garantisini verir.



sealed (Eksiksizlik Denetimi)

Bir alt tür ailesinde tüm olasılıkların önceden bilinmesini sağlar. Switch yapılarında derleyiciye tüm durumların kapsandığını garanti eder.



Gemini Notebook

Özet: Sistem Mimarının Vizyonu

"İyi bir yazılım mühendisi sadece çalışan kod yazmaz, sistem tasarlar."

Sağlam Temeller: base, sealed veya final kullanarak kalıtım kurallarını kesin bir şekilde çizer.

Kod Zekası ve Sensors: Metadatalar (@) ile kodu belgelendirir, derleyiciyi yönlendirir ve sınırları korur.



Sağlam Temeller: base, sealed veya final kullanarak kalıtım kurallarını kesin bir şekilde çizer.

Modüler Organizasyon: Kütüphaneleri izole eder (_) ve tembel yüklemeyle (Deferred Loading) performansı optimize eder.

Dart 3.13, temiz, hafif ve kontrol edilebilir bir mimari inşa etmeniz için gereken tüm araçları sağlar.

Gemini Notebook